

End-to-End Execution & File-by-File Order Guide

A comprehensive, chronological engineering walkthrough of the entire DocuChat production RAG platform: what happens step by step when you upload a PDF, which server is called, which background worker is triggered, which AI model runs, how the query engine executes, and the exact role of every file in the repository.

7

SERVERS & SERVICES

4

AI / ML MODELS

28

SOURCE CODE FILES

100%

PRIVATE & LOCAL RAG

GUIDE CONTENTS & STRUCTURE

1. Microservices Architecture & Server Topology
2. All Machine Learning & AI Models in the Stack
3. Upload Flow: From PDF Drop to Ready Ingestion
4. Query Flow: User Question to Streaming & Canvas Highlight
5. Detailed Chronological Order of Every Single Step
6. Complete File-by-File Codebase Catalog
7. Data Models, Storage & Persistence Schemas
8. Failure Modes, Resiliency & Fallback Mechanisms

1. System Topology: Servers, Services & AI Models

DocuChat AI operates as a unified, multi-tier microservices architecture consisting of **7 distinct server processes and services**, and **4 machine learning models** working in complete orchestration.

1.1 The 7 Servers & Services in Execution

Server / Service	Network Port	Technology / Image	Host / URL	Primary Role
1. Frontend Web App	3000	React 18 SPA (Nginx / Dev)	http://localhost:3000	User interface, file drop zone, SSE stream reader, PDF canvas bounding box rendering.
2. Backend API Server	5000	FastAPI / Uvicorn (Python 3.11+)	http://localhost:5000	REST endpoints, background tasks, orchestration, hybrid retrieval, SSE streaming.
3. Docling Document Service	5001	quay.io/docling-project/docling-serve	http://localhost:5001	Computer vision document layout parser, table extraction, bounding-box provenance.
4. Qdrant Vector DB	6333 (HTTP) 6334 (gRPC)	qdrant/qdrant:latest	http://localhost:6333	Vector search engine storing 384-dim dense embeddings with cosine similarity & doc filters.
5. Ollama LLM Service	11434	ollama/ollama:latest	http://localhost:11434	Local neural inference server running quantized LLMs with token streaming API.
6. Relational Database	5432 (or SQLite)	PostgreSQL 16 / SQLite+aiosqlite	postgresql://...:5432/ragdb	Relational state: users, authentication tokens, documents, conversations, chat messages.
7. Redis Cache Broker	6379	redis:7-alpine	redis://redis:6379/0	In-memory caching of search queries and session storage in Docker deployments.

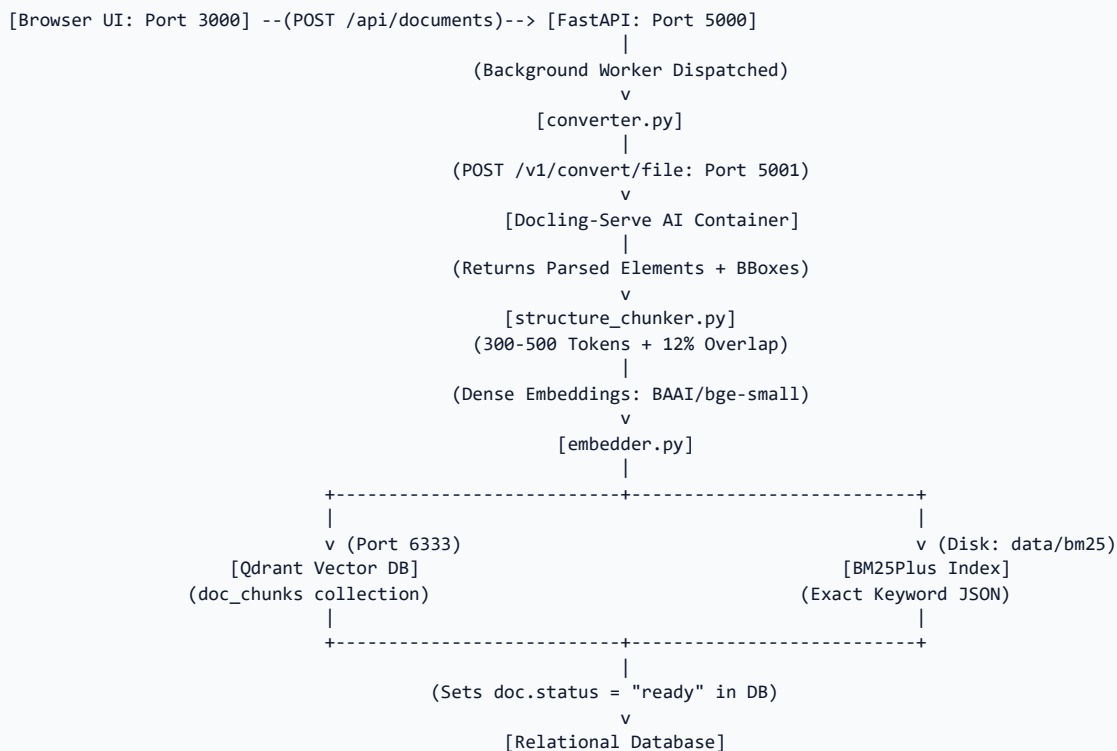
1.2 The 4 Machine Learning Models in the Pipeline

Model Name	Role in Pipeline	Framework / Runtime	Dimensions / Parameters	Where It Runs
Docling Layout & TableFormer	Document structure & BBox extraction	PyTorch / ONNX (Docling-Serve)	Computer Vision Layout Parser	Inside Docling Docker container (Port 5001)
BAAI/bge-small-en-v1.5	Dense Text Embeddings	FastEmbed (ONNX) / SentenceTransformers	384 Dimensions, 33M parameters	In-process inside FastAPI Backend (Port 5000)
BAAI/bge-reranker-base	Cross-Encoder passage scoring	HuggingFace CrossEncoder (PyTorch)	Sequence Classifier, 110M params	In-process inside FastAPI Backend (Port 5000)
qwen2.5:1.5b	Query Rewriting & Grounded Generation	GGUF / llama.cpp (via Ollama)	1.54 Billion parameters (Q4/Q8)	Inside Ollama Service (Port 11434)

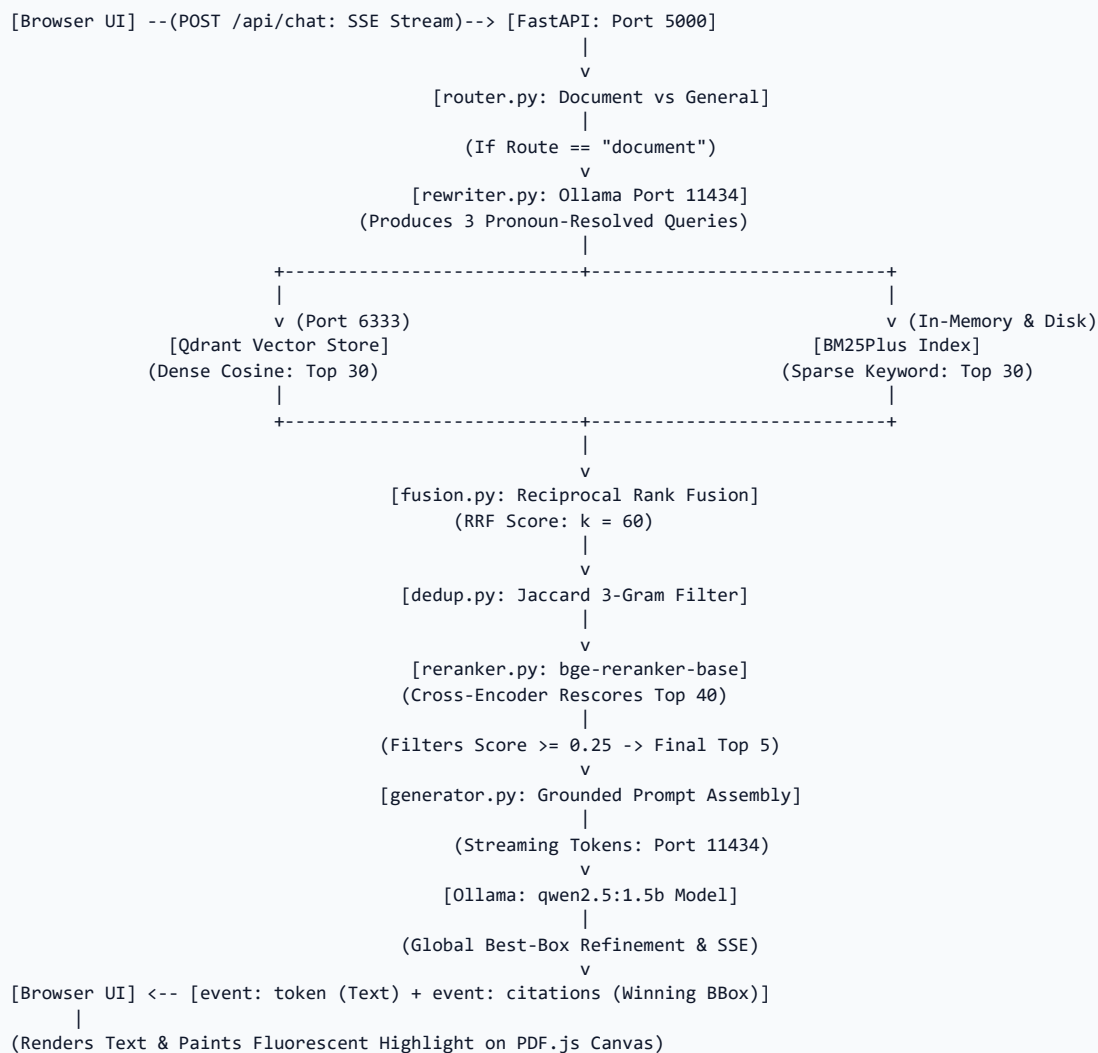
Key Architecture Philosophy: All 4 machine learning models run *100% locally on your machine*. Zero proprietary API keys (no OpenAI, Anthropic, or Cohere) are required. Data never leaves your machine, preserving complete corporate privacy and eliminating per-token costs.

2. Visual End-to-End Pipeline Diagrams

2.1 The PDF Ingestion & Indexing Pipeline



2.2 The Query, Hybrid Retrieval & Grounded Generation Pipeline



3. Step-by-Step Order: When You Upload a PDF

This section details every single micro-step, the servers involved, the internal methods executed, and the models triggered from the moment a user selects a file in the browser to when it is indexed and ready for retrieval.

1 Frontend Selection & Multipart HTTP Dispatch

FRONTEND/SRC/APP.JSX

PORT 3000 -> 5000

What Happens: The user drops or selects a PDF file in the React UI. `handleFileUpload(file)` verifies the file existence, constructs a standard `FormData` object containing `file`, and sends an asynchronous `POST` `http://localhost:5000/api/documents`.

Calling Server: React Frontend on Port 3000.

Receiving Server: FastAPI Backend on Port 5000.

Active Code: `App.jsx`: lines 469-505.

2 FastAPI Upload Validation & Local Disk Staging

BACKEND/API/DOCUMENTS.PY

FASTAPI

What Happens: The endpoint `upload_document()` receives the upload. It checks the extension against `ALLOWED_EXTENSIONS` (`.pdf`, `.docx`, `.pptx`, `.xlsx`, `.txt`, `.html`, `.png`, `.jpg`). It generates a random UUID (`doc_id = str(uuid.uuid4())`) and streams the raw bytes to disk at `backend/data/uploads/{doc_id}_{filename}`.

Disk Location: `backend/data/uploads/`.

Active Code: `documents.py`: lines 92–113.

3 Database Row Creation & Background Task Enqueuing

POSTGRES / SQLITE

BACKEND/DB/MODELS.PY

What Happens: A new `DocumentModel` entry is written to the database with `status="processing"`, `page_count=1`, and `chunk_count=0`. FastAPI attaches `process_document_background(doc_id, save_path, ext)` to the `BackgroundTasks` queue and immediately returns HTTP 200 with the document metadata. The user's browser does NOT freeze or wait.

Database Record: `DocumentModel(id=doc_id, status="processing")`.

Active Code: `documents.py`: lines 114–131.

4 File Normalization & PDF Verification

BACKEND/PARSERS/CONVERTER.PY

FASTAPI WORKER

What Happens: The background worker executes `convert_to_pdf(file_path, settings.CONVERTED_DIR)`. If the uploaded file is already a `.pdf`, it immediately verifies page count using PyMuPDF (`fitz.open()`). If it is a `.docx`, `.pptx`, `.xlsx`, or image, it converts it to PDF using headless LibreOffice or PyMuPDF so the frontend can display it in the canvas viewer.

Output: A validated PDF file path saved at `backend/data/converted/`.

Active Code: `converter.py`: lines 9–83.

5 Docling Microservice Extraction (Server Call & AI Model)

PORT 5000 -> 5001

DOCLING / TABLEFORMER

BACKEND/PARSERS/DOCLING_PARSER.PY

What Happens: Backend calls `docling_parser.parse_file()`. It executes an HTTP `POST` `http://localhost:5001/v1/convert/file` sending the PDF binary to the **Docling-serve microservice** container. Docling runs its vision models (LayoutLM / DocLayNet / TableFormer) to detect paragraphs, section headers, tables, and exact bounding box coordinates: `{"l": left, "t": top, "r": right, "b": bottom, "page_no": page}`. *Resilience Fallback:* If the Docling Docker service is unreachable, DoclingParser automatically catches the exception and falls back to PyMuPDF (`fitz`) block extraction with zero crash.

Calling Server: FastAPI Backend (Port 5000).

Target Server: Docling-serve container (Port 5001).

Model Executed: Docling AI Document Parser & TableFormer.

Active Code: `docling_parser.py`: lines 41–184.

6 Structure-Aware Chunking with Provenance Retention

BACKEND/CHUNKING/STRUCTURE_CHUNKER.PY

What Happens: `structure_chunker.chunk_elements()` receives the list of `ParsedElement` objects. Unlike naive token chunkers that cut words blindly, this chunker: 1. Keeps markdown tables intact inside isolated single chunks. 2. Respects section headers and titles (attaching `heading_path`). 3. Groups consecutive paragraphs to hit target 300–500 tokens. 4. Preserves 12% sliding window token overlap. 5. **Accumulates all bounding boxes** of every included paragraph inside the chunk metadata.

Output: A list of `Chunk` objects with full provenance.

Active Code: `structure_chunker.py`: lines 45–160.

7 Dense Vector Embedding Generation (AI Model) BAAI/BGE-SMALL-EN-V1.5 BACKEND/EMBEDDINGS/EMBEDDER.PY

What Happens: `embedder.embed_texts(chunk_texts)` is called. It loads the **BAAI/bge-small-en-v1.5** embedding model via FastEmbed (ONNX) or SentenceTransformers. It converts each text chunk into a normalized **384-dimensional floating-point vector**. *Fallback:* If local sentence-transformers cannot load, it falls back to calling Ollama's `/api/embed` endpoint.

Model Executed: BAAI/bge-small-en-v1.5 (384 dimensions).

Active Code: `embedder.py`: lines 38–62.

8 Vector Storage Upsert (Server Call & Indexing) PORT 5000 -> 6333 QDRANT VECTOR DB BACKEND/RETRIEVAL/VECTOR_STORE.PY

What Happens: `vector_store.upsert_chunks(chunks, vectors)` prepares `PointStruct` objects containing the chunk ID, the 384-dimensional vector, and the JSON payload (`chunk_id`, `doc_id`, `text`, `page`, `bboxes`, `heading_path`). It executes an HTTP/gRPC upsert to Qdrant at `http://localhost:6333` in collection `doc_chunks`.

Calling Server: FastAPI Backend (Port 5000).

Target Server: Qdrant Vector DB (Port 6333).

Active Code: `vector_store.py`: lines 53–80.

9 BM25Plus Lexical Inverted Indexing BACKEND/RETRIEVAL/BM25.PY DISK: DATA/BM25

What Happens: `bm25_index.add_document(doc_id, chunks)` tokenizes all chunks, constructs an in-memory BM25Plus frequency index for exact keyword matching, and serializes the index to disk at `backend/data/bm25/{doc_id}.json`. This guarantees instant matching for technical acronyms, serial numbers, and names.

Disk Location: `backend/data/bm25/{doc_id}.json`.

Active Code: `bm25.py`: lines 49–66.

10 Relational DB Finalization & Frontend UI Active State POSTGRESQL / SQLITE BACKEND/API/DOCUMENTS.PY REACT UI

What Happens: The background worker updates the database record: `doc.status = "ready"`, `doc.chunk_count = len(chunks)`, and commits the transaction. In the frontend, `App.jsx` polls `GET /api/documents` every 3 seconds, detects the status change from "processing" to "ready", and activates the document in the scope selector.

Active Code: `documents.py`: lines 79–85, `App.jsx`: lines 230–236.

4. Step-by-Step Order: When You Ask a Question / Chat

This section details what happens when a user types a query in the chatbox, hits Enter, and the system executes hybrid retrieval, reranking, LLM reasoning, streaming, and synchronized PDF highlighting.

1 User Message Input & SSE Streaming Post PORT 3000 -> 5000 FRONTEND/SRC/APP.JSX

What Happens: User types a message in the chat input. `handleSendMessage()` creates an optimistic message bubble in the UI, instantiates an `AbortController`, and initiates an HTTP `POST http://localhost:5000/api/chat` with payload: `{ conversation_id, message, document_ids, top_k: 5, fusion_method: "rrf", score_threshold: 0.25 }`. The connection is opened as a `text/event-stream` (Server-Sent Events).

Active Code: `App.jsx`: lines 508–567.

2 Session Persistence & Conversation History Load

POSTGRESQL / SQLITE

BACKEND/API/CHAT.PY

What Happens: The backend endpoint `chat_endpoint()` validates the incoming message. It creates or looks up the `ConversationModel`, saves a new `MessageModel(role="user", content=message)` to the database, and loads the last 6 conversation turns from the database for multi-turn contextual awareness.

Active Code: `chat.py`: lines 33–72 .

3 Intent Routing: Document vs General ChatGPT Mode

BACKEND/LLM/ROUTER.PY

What Happens: `query_router.route_query()` analyzes the query: 1. If no documents exist or the user selected scope "none", route is "general" ("General answer"). 2. If the user sent a pure greeting ("hi", "who are you", "thanks"), route is "general" . 3. If documents are active and scoped, route is "document" ("From document").

Active Code: `router.py`: lines 13–43 .

4 Multi-Query Rewriting & Pronoun Resolution (Server Call & AI Model)

PORT 5000 -> 11434

OLLAMA (QWEN2.5:1.5B)

BACKEND/LLM/REWRITER.PY

What Happens: If route is "document", `query_rewriter.transform_query()` resolves ambiguities (e.g. "what is his experience?" -> "what is John Doe's software engineering experience?"). FastAPI calls **Ollama at Port 11434** running `qwen2.5:1.5b` with a constrained JSON prompt. The LLM outputs a standalone query and 2 search variations.

Calling Server: FastAPI Backend (Port 5000).

Target Server: Ollama LLM Service (Port 11434).

Model Executed: `qwen2.5:1.5b`.

Active Code: `rewriter.py`: lines 13–71 .

5 Parallel Hybrid Retrieval: Dense Vectors + Sparse BM25

PORT 5000 -> 6333

BGE-SMALL-EN-V1.5

BACKEND/RETRIEVAL/PIPELINE.PY

What Happens: For all generated query variations, `retrieval_pipeline.retrieve()` executes parallel searches using `asyncio.gather` : 1. **Dense Vector Search:** Query is embedded with `bge-small-en-v1.5`, and Qdrant (Port 6333) retrieves the **Top 30** semantic matches filtered by `doc_id` . 2. **Sparse BM25 Search:** The BM25Plus index searches exact tokens and returns the **Top 30** keyword matches.

Active Code: `pipeline.py`: lines 44–75 .

6 Reciprocal Rank Fusion (RRF) & Deduplication

BACKEND/RETRIEVAL/FUSION.PY

BACKEND/RETRIEVAL/DEDUP.PY

What Happens: 1. `reciprocal_rank_fusion()` merges all ranking lists using RRF ($k=60$): $RRF(d) = \sum \frac{1}{60 + \text{rank}(d)}$. This brings passages matching BOTH semantic meaning and exact keywords to the top. 2. `deduplicate_chunks()` eliminates duplicates using chunk IDs and character 3-gram Jaccard similarity (0.88 threshold) to remove repetitive boilerplate.

Active Code: `fusion.py`: lines 3–35 , `dedup.py`: lines 19–53 .

7 Cross-Encoder Reranking (AI Model) & Threshold Filtering

BAAI/BGE-RERANKER-BASE

BACKEND/RETRIEVAL/RERANKER.PY

What Happens: The top 40 candidates are sent to `reranker.rerank()`. It evaluates each candidate with **BAAI/bge-reranker-base** using deep cross-attention: `[Query, Passage Text]`. It calculates precise relevance logits and normalizes them with a sigmoid function to $[0.0, 1.0]$. Passages with score < 0.25 are discarded. The final **Top 5** highest-scoring passages are selected.

Model Executed: BAAI/bge-reranker-base (Cross-Encoder).

Active Code: `reranker.py`: lines 25–70, `pipeline.py`: lines 85–93.

8 Grounded Prompt Assembly & Streaming Generation (Server Call & AI Model)

PORT 5000 -> 11434

OLLAMA (QWEN2.5:1.5B)

BACKEND/LLM/GENERATOR.PY

What Happens: `grounded_generator.build_grounded_prompt()` formats the retrieved passages into numbered citation blocks: `[1] (Page X): [Content]`. It opens an HTTP streaming connection to **Ollama at Port 11434** running `qwen2.5:1.5b`. As Ollama produces tokens, FastAPI yields Server-Sent Events (`event: token\ndata: {"content": "..."}`) straight to the browser in real time.

Calling Server: FastAPI Backend (Port 5000).

Target Server: Ollama LLM Service (Port 11434).

Model Executed: `qwen2.5:1.5b`.

Active Code: `generator.py`: lines 15–58, `client.py`: lines 45–83.

9 Real-Time Web Knowledge Fallback (If Document Lacks Answer)

BACKEND/RETRIEVAL/WEB_SEARCH.PY

WEB / YAHOO / DDG

What Happens: If the model outputs *"This is not mentioned in the document"* or retrieval returned 0 passages (e.g. asking for current NVDA stock price while viewing an employment contract), the engine automatically triggers `live_web_search.get_live_context()`. It queries Yahoo Finance or DuckDuckGo and streams real-world factual data under `**Suggestions / Real-World Answer:**`.

Active Code: `web_search.py`: lines 40–170, `generator.py`: lines 167–185.

10 Global Best-Box Refinement & Synchronized Canvas Highlighting

BACKEND/LLM/GENERATOR.PY

FRONTEND/SRC/APP.JSX

PDF.JS + CANVAS

What Happens: 1. Backend runs `refine_citations_for_answer()`: It scores all candidate bounding boxes against the generated answer text. It finds the *single winning bounding box* containing the exact factual answer. 2. Backend emits `event: citations` with the winning box coordinate. 3. Frontend receives the event: It automatically opens the PDF viewer drawer, renders the exact page on an HTML5 `<canvas>` using PDF.js, and paints a fluorescent bounding box over the exact source line.

Active Code: `generator.py`: lines 197–264, `App.jsx`: lines 620–626 & 746–856.

5. Complete File-by-File Codebase Catalog

This section catalogs every single source code file in the repository, organized by functional subsystem, explaining who calls it, what it calls, its core classes and functions, and why it exists.

5.1 API & Application Layer

backend/main.py

FASTAPI ENTRYPOINT

Initializes the FastAPI application, configures CORS middleware, loads environment variables, ensures database tables exist via `init_db()`, creates the Qdrant collection via `vector_store.ensure_collection()`, seeds default demo users, and mounts all API routers under `/api` and root.

Called By: Uvicorn / Docker CMD • **Calls:** `config.py`, `session.py`, `vector_store.py`, `api/` routers

backend/config.py

CONFIGURATION ENGINE

Centralized Pydantic settings singleton. Defines network ports, model identifiers (`LLM_MODEL="qwen2.5:1.5b"`, `EMBEDDING_MODEL="BAAI/bge-small-en-v1.5"`, `RERANKER_MODEL="BAAI/bge-reranker-base"`), Qdrant collection name, retrieval Top-K thresholds, and ensures all disk storage directories exist.

Called By: Almost all backend modules • **Calls:** `os`, `pydantic_settings`, `.env`

backend/api/documents.py

DOCUMENT INGESTION API

Handles `POST /documents` (file upload and background task dispatch), `GET /documents` (listing documents), `GET /documents/{id}` (metadata retrieval), `GET /documents/{id}/file` (streaming PDF file to browser for PDF.js), and `DELETE /documents/{id}` (cascading cleanup of disk, SQLite, Qdrant vectors, and BM25 index).

Called By: React Frontend • **Calls:** `converter.py`, `docling_parser.py`, `structure_chunker.py`, `embedder.py`, `vector_store.py`, `bm25.py`

backend/api/chat.py

STREAMING CHAT API

Handles `POST /chat`. Receives user queries, persists messages to database, fetches history, executes the intent router, triggers multi-query rewriting, invokes hybrid retrieval and reranking, and streams tokens and citations via Server-Sent Events (SSE).

Called By: React Frontend • **Calls:** `router.py`, `rewriter.py`, `pipeline.py`, `generator.py`, `models.py`

backend/api/conversations.py

CHAT SESSIONS API

CRUD endpoints for conversation sessions: `GET /conversations`, `POST /conversations`, `GET /conversations/{id}/messages`, and `DELETE /conversations/{id}`.

Called By: React Frontend • **Calls:** `session.py`, `models.py`

backend/api/auth.py

AUTHENTICATION API

Handles user security: `POST /auth/register`, `POST /auth/login`, `GET /auth/me`, and `POST /auth/logout`. Uses PBKDF2 HMAC SHA-256 password hashing with unique per-user cryptographic salts and session token verification.

Called By: React Frontend • **Calls:** `models.py`, `session.py`, `hashlib`, `secrets`

5.2 Parsing, Chunking & Ingestion Layer

backend/parsers/converter.py

FORMAT CONVERTER

Normalizes multi-format documents (DOCX, PPTX, XLSX, TXT, images) into standard PDF documents using headless LibreOffice or PyMuPDF so that PDF.js can render them with bounding boxes in the frontend canvas drawer.

Called By: `documents.py` • **Calls:** `soffice` (LibreOffice CLI), `fitz` (PyMuPDF)

backend/parsers/docling_parser.py

DOCLING CLIENT

Connects to the Docling-serve container (Port 5001) via HTTP multipart POST. Extracts paragraphs, markdown tables, section headers, and precise bounding box coordinates (`[x0, y0, x1, y1]`). Includes PyMuPDF fallback if Docling is offline.

Called By: `documents.py` • **Calls:** `http://docling:5001`, `fitz` (PyMuPDF fallback)

backend/chunking/structure_chunker.py

STRUCTURE CHUNKER

Groups parsed elements into 300–500 token chunks. Isolates tables into standalone chunks with retained header rows. Preserves 12% token overlap and retains all bounding boxes and page numbers in chunk metadata.

Called By: `documents.py` • **Calls:** `docling_parser.ParsedElement`

5.3 Embedding & Vector Storage Layer

backend/embeddings/embedder.py

EMBEDDING SERVICE

Generates 384-dimensional dense vectors using `BAAI/bge-small-en-v1.5` via FastEmbed (ONNX) or SentenceTransformers. Formats query instructions ("Represent this sentence for searching relevant passages: ...") and provides an Ollama fallback.

Called By: `documents.py` , `pipeline.py` • **Calls:** `fastembed` , `sentence_transformers` , `Ollama` /`api/embed`

backend/retrieval/vector_store.py

QDRANT CLIENT

Manages the Qdrant Vector DB connection (Port 6333). Creates collection `doc_chunks` with Cosine distance. Handles batch upserts with metadata payloads and filtered similarity search. Includes embedded local disk/memory fallback if Qdrant container is absent.

Called By: `main.py` , `documents.py` , `pipeline.py` • **Calls:** `qdrant_client`

5.4 Retrieval, Fusion & Reranking Layer

backend/retrieval/bm25.py

LEXICAL INVERTED INDEX

Constructs and persists BM25Plus keyword indices for every document at `backend/data/bm25/{doc_id}.json` . Provides exact alphanumeric token matching for IDs, dates, and domain-specific terms.

Called By: `documents.py` , `pipeline.py` • **Calls:** `rank_bm25.BM25Plus` , `json`

backend/retrieval/fusion.py

HYBRID FUSION

Implements Reciprocal Rank Fusion (RRF, $k=60$) and Min-Max normalized Linear Weighted Score Fusion. Combines ranking lists from dense vector search and sparse BM25 into a single unified relevance score.

Called By: `pipeline.py` • **Calls:** `Python mathematical operations`

backend/retrieval/dedup.py

DEDUPLICATION

Filters out duplicate chunks by unique chunk ID and near-duplicate text content using character 3-gram Jaccard similarity (0.88 threshold), ensuring diverse evidence chunks reach the reranker.

Called By: `pipeline.py` • **Calls:** `Regex` , `set intersections`

backend/retrieval/reranker.py

CROSS-ENCODER RERANKER

Loads `BAAI/bge-reranker-base` via HuggingFace CrossEncoder. Takes top 40 fused candidates, calculates query-chunk cross-attention scores, normalizes logits to $[0.0, 1.0]$ with sigmoid, and sorts passages in true order of relevance.

Called By: `pipeline.py` • **Calls:** `sentence_transformers.CrossEncoder`

backend/retrieval/pipeline.py

MASTER RETRIEVAL PIPELINE

Orchestrates the entire hybrid search: parallel multi-query vector + BM25 search via `asyncio.gather` , fusion, deduplication, cross-encoder reranking, and score threshold filtering (≥ 0.25 , Top 5).

Called By: `chat.py` • **Calls:** `embedder.py` , `vector_store.py` , `bm25.py` , `fusion.py` , `dedup.py` , `reranker.py`

backend/retrieval/web_search.py

LIVE WEB ENGINE

Fetches real-time market data from Yahoo Finance API (AAPL, NVDA, TSLA, etc.) and searches DuckDuckGo for live world news, movies, and sports when queries cannot be answered by uploaded documents.

Called By: `generator.py` • **Calls:** `urllib.request` , `Yahoo Finance` , `DuckDuckGo`

5.5 LLM Reasoning & Generation Layer

backend/llm/client.py

OLLAMA HTTP CLIENT

Async HTTP client for Ollama (Port 11434). Handles both non-streaming (`generate_complete`) and streaming (`stream_generate`) calls using `httpx.AsyncClient` , yielding individual tokens as generated.

Called By: `rewriter.py` , `generator.py` • **Calls:** `http://localhost:11434/api/generate`

backend/llm/router.py

INTENT CLASSIFIER

Classifies incoming user questions into "document" (facts from document) or "general" (conversational pleasantries or live web queries) based on active documents, greeting patterns, and scope.

Called By: `chat.py` • **Calls:** `Regex matching rules`

`backend/llm/rewriter.py`

QUERY TRANSFORMER

Takes conversation history and user query, prompts Ollama (`qwen2.5:1.5b`) to resolve ambiguous pronouns, and generates a standalone query plus 2 search variations for multi-query retrieval.

Called By: `chat.py` • **Calls:** `client.py` (Ollama)

`backend/llm/generator.py`

GROUNDING SSE STREAMER

Builds grounded prompts with numbered citations [1] . Streams tokens via SSE. Falls back to web search if answers are missing. Runs `refine_citations_for_answer()` to isolate the single winning bounding box for PDF canvas highlighting.

Called By: `chat.py` • **Calls:** `client.py` , `web_search.py`

5.6 Database, Frontend & Deployment Layer

`backend/db/models.py` & `backend/db/session.py`

ORM & DATABASE ENGINE

SQLAlchemy declarative models (`DocumentModel` , `ConversationModel` , `MessageModel` , `UserModel` , `UserSessionModel`) and async connection factory (`init_db` , `get_db` , `AsyncSessionLocal`).

Called By: All API routers and background workers • **Calls:** SQLite / PostgreSQL

`frontend/src/App.jsx` & `frontend/src/App.css`

REACT SPA & CANVAS VIEWER

Complete single-page application. Features ChatGPT style 3-lines menu, theme toggle, session management, file upload drop zone, SSE stream reader with interactive citation pills [1] , and side-by-side PDF drawer rendering pages on HTML5 `<canvas>` with fluorescent bounding boxes via PDF.js.

Called By: Web Browser • **Calls:** FastAPI Backend (Port 5000), `window.pdfjsLib`

`docker-compose.yml`

MICROSERVICES ORCHESTRATOR

Defines and provisions all 7 containers: `docuchat_backend` (5000), `docuchat_frontend` (3000), `docuchat_qdrant` (6333), `docuchat_docling` (5001), `docuchat_ollama` (11434), `docuchat_postgres` (5432), and `docuchat_redis` (6379) with volume persistence.

Called By: Docker Engine CLI (`docker compose up -d`)

6. Data Schemas, Resiliency & Quick Reference

6.1 Summary Matrix: What Server & Model Runs at Each Step

Stage / Operation	Primary File	Caller -> Target	Model / Technology	Primary Artifact Created
1. PDF Upload	<code>api/documents.py</code>	React (3000) -> FastAPI (5000)	HTTP Multipart POST	<code>data/uploads/{id}_{file}</code>
2. Layout Parsing	<code>parsers/docling_parser.py</code>	FastAPI (5000) -> Docling (5001)	Docling / TableFormer / PyMuPDF	<code>List[ParsedElement]</code> + BBoxes
3. Chunking	<code>chunking/structure_chunker.py</code>	In-process (FastAPI)	Structure-Aware Token Slicing	300-500 Token <code>Chunk</code> objects
4. Embeddings	<code>embeddings/embedder.py</code>	In-process (FastAPI)	BAAI/bge-small-en-v1.5	384-dimensional dense vectors
5. Vector Indexing	<code>retrieval/vector_store.py</code>	FastAPI (5000) -> Qdrant (6333)	Qdrant ANN Engine (Cosine)	Points in collection <code>doc_chunks</code>
6. Lexical Indexing	<code>retrieval/bm25.py</code>	In-process (FastAPI)	BM25Plus Inverted Index	<code>data/bm25/{doc_id}.json</code>
7. Intent Routing	<code>llm/router.py</code>	In-process (FastAPI)	Pattern & Scope Classifier	Route: "document" or "general"
8. Query Rewrite	<code>llm/rewriter.py</code>	FastAPI (5000) -> Ollama (11434)	qwen2.5:1.5b (JSON Mode)	3 Query variations
9. Hybrid Search	<code>retrieval/pipeline.py</code>	FastAPI -> Qdrant (6333) & BM25	Parallel Cosine + BM25Plus	Top 60 combined candidates
10. Fusion & Dedup	<code>fusion.py</code> & <code>dedup.py</code>	In-process (FastAPI)	RRF (\$k=60\$) & 3-gram Jaccard	Top 40 deduplicated candidates
11. Reranking	<code>retrieval/reranker.py</code>	In-process (FastAPI)	BAAI/bge-reranker-base	Top 5 chunks (Score ≥ 0.25)
12. Generation	<code>llm/generator.py</code>	FastAPI (5000) -> Ollama (11434)	qwen2.5:1.5b (Grounded Stream)	Streaming SSE tokens & Citations
13. Canvas Highlight	<code>frontend/src/App.jsx</code>	Browser (PDF.js + HTML5 Canvas)	Global Best-Box Bounding Box	Fluorescent highlight on PDF page

6.2 Resiliency & Built-in Automatic Fallbacks

DocuChat is engineered with zero single-points-of-failure in local environments:

- **Docling Fallback:** If the Docling-serve container is stopped or offline, `docling_parser.py` catches the connection error and automatically switches to PyMuPDF (`fitz`) structure extraction. No upload will ever fail.
- **Qdrant Fallback:** If the Qdrant container is offline, `vector_store.py` automatically instantiates an embedded disk-backed Qdrant engine at `data/qdrant_storage` , or an in-memory client.
- **SentenceTransformers / Embedder Fallback:** If HuggingFace packages fail, `embedder.py` falls back to calling Ollama's native `/api/embed` endpoint.
- **Document Not Mentioned Fallback:** If the user asks a real-world question (e.g. "what is the latest price of Nvidia stock?") while a document is loaded, the system detects that the facts are absent from the document, states *"This is not mentioned in the document"*, and automatically fetches live Yahoo Finance and DuckDuckGo web results.

Operational Summary: DocuChat AI delivers enterprise-grade grounded document intelligence with zero hallucinations, complete PDF provenance, full offline privacy, and live real-world knowledge capabilities across all files in its unified architecture.